APPLIED DATA SCIENCE CAPSTONE

Predicting Falcon 9 First-Stage Landing Success

A data science analysis of SpaceX launch outcomes — from data collection through predictive modeling

Abdou A. Diakite
IBM Data Analyst Certificate
August 11, 2026
Github:

BACKGROUND

Introduction

Project Background

- SpaceX advertises Falcon 9 launches at ~$62M, far below competitors' ~$165M+, largely because SpaceX reuses the first stage instead of discarding it.
- Reuse is only possible when the first stage successfully lands, so landing success directly determines launch cost.
- A competing company bidding against SpaceX would benefit from estimating this cost — which requires predicting landing success.

Problem Statement

Can we predict whether the Falcon 9 first stage will land successfully, using publicly available launch data — payload mass, orbit type, launch site, booster version, and more?

Key Questions Explored

- Which variables most influence landing outcome?
- How has success rate changed over time?
- Which classification model predicts outcome best?

OVERVIEW

Executive Summary

- This project aims to predict whether the Falcon 9 first stage will land successfully so that a competing company can make more informed launch bids.
- Methods used: data collection (API + web scraping), data wrangling, exploratory data analysis, interactive visual analytics and predictive classification modeling.
- After comparing multiple models, we find that the Falcon 9 booster has a high probability of successful landing (87.7% based on decision tree classifier). Success is highly correlated with launch site, orbits, boosters and payload mass.
- Success rate has reached 90% in recent years, thanks to new generation of boosters.

87.7%

Best model accuracy
Decision Tree classifier

90

Launch records analyzed
Falcon 9 launches (2010–2020)

66.7%

Overall landing success rate
60 of 90 landings succeeded

SpaceX REST API

1 GET request to api.spacexdata.com/v4/launches/past

2 Parse JSON response into a pandas DataFrame

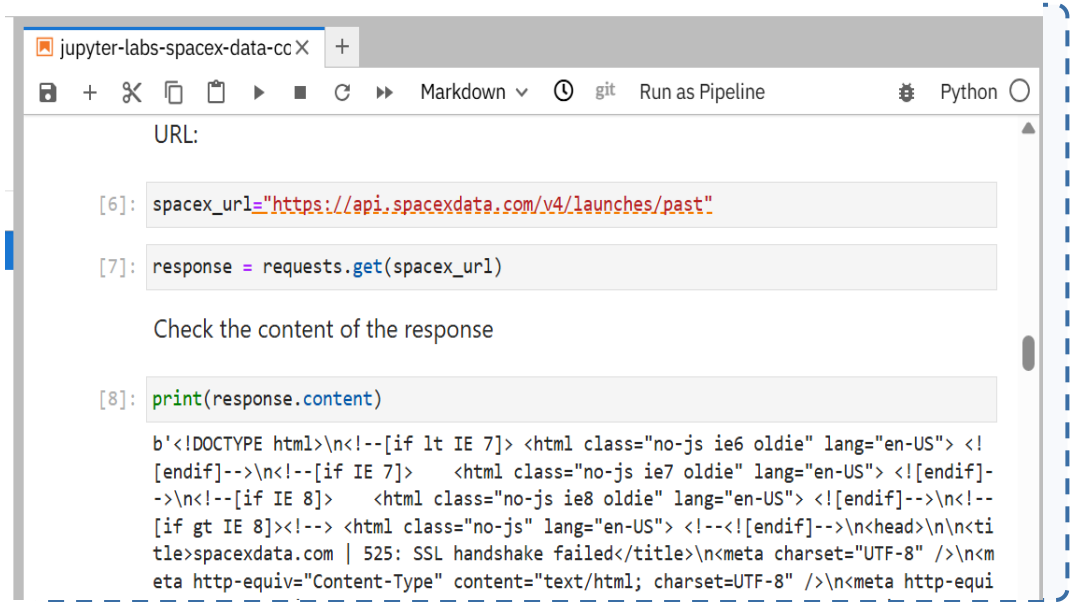
3 Use rocket, payload, launchpad, core IDs to make follow-up GET requests for details

4 Merge nested results into flat columns per launch

5 Filter to Falcon 9 launches only (drop Falcon 1)

Key Steps

- requests.get(url).json() against the SpaceX v4 API
- IDs returned per launch (rocket, payloads, launchpad, cores) require separate GET calls to resolve into readable fields
- Custom functions (e.g. getBoosterVersion(), getLaunchSite()) map IDs → names and append to lists
- Combine into a single DataFrame; export to spacex_api.csv



Notebook / Code: *github.com/your-username/your-repo-name*

DATA COLLECTION

Web Scraping (Wikipedia)

1 GET the Wikipedia “List of Falcon 9 and Falcon Heavy launches” page

2 Parse HTML with BeautifulSoup

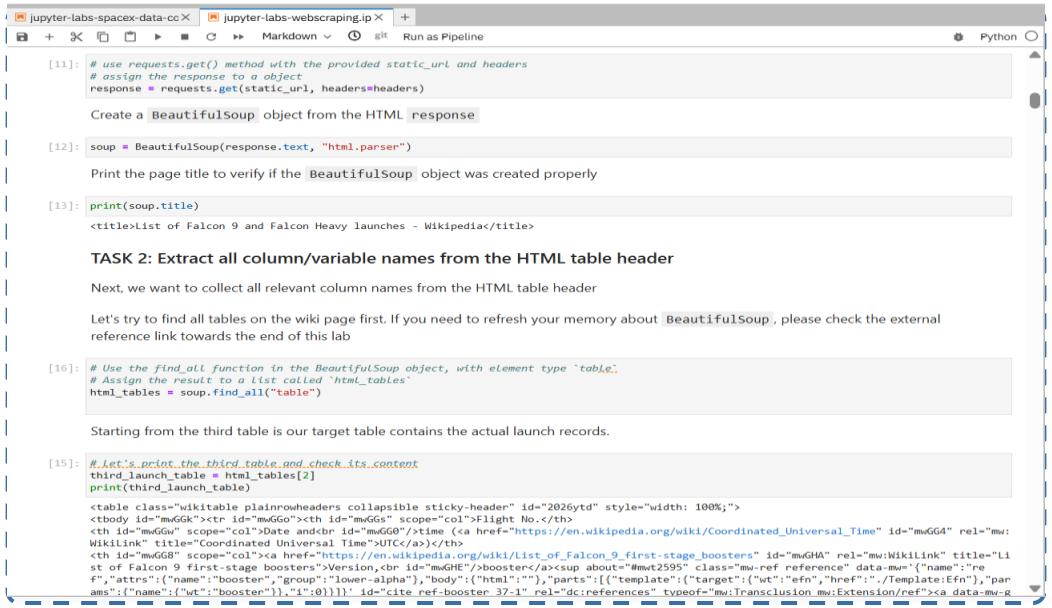
3 Locate the launch-record HTML table

4 Extract each column (date, booster, site, payload, outcome)

5 Assemble rows into a DataFrame

Key Steps

- requests.get() the Wikipedia revision URL, then BeautifulSoup(html, 'html.parser')
- soup.find_all('table', 'wikitable plainrowheaders collapsible') to locate the launch table
- Custom parsing functions per column (date/time, booster version, payload mass, orbit, outcome)
- Append parsed cells into a dict of lists → pd.DataFrame; export to spacex_web_scraped.csv



Notebook / Code: *github.com/your-username/your-repo-name*

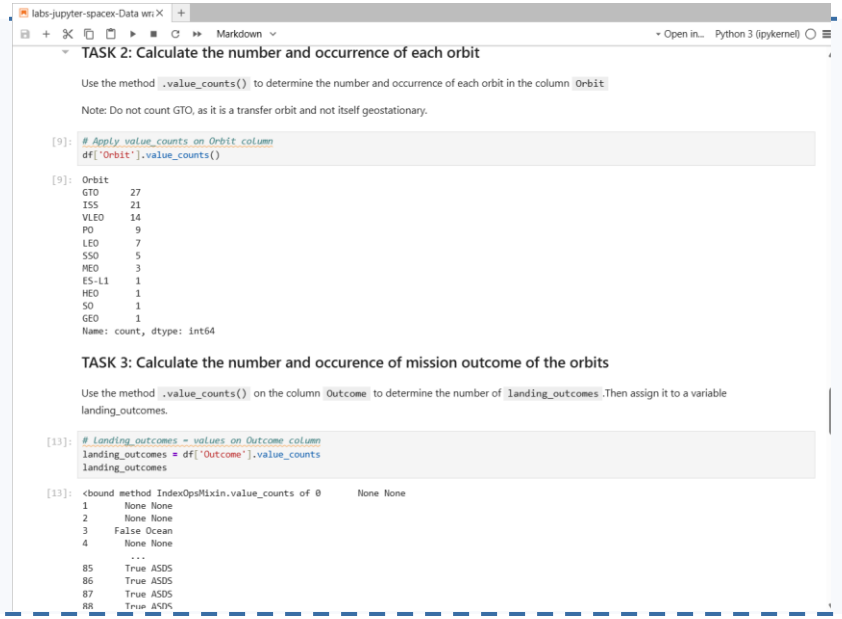
PREPARATION

Data Wrangling Methodology

1. Combine API + scraped datasets; resolve column mismatches
2. Handle missing values (e.g. impute mean payload mass)
3. Standardize categorical labels (site, orbit, outcome text)
4. Engineer the binary Class label (1 = success, 0 = failure) from landing outcome text
5. Export the clean, model-ready dataset

What Counts as a "Successful Landing"

- Outcome strings like 'True ASDS', 'True RTLS', 'True Ocean' → Class = 1
- 'False ASDS', 'False RTLS', 'None None' → Class = 0
- This binary label becomes the target variable for the classification models



Notebook / Code: *github.com/your-username/your-repo-name*

EXPLORATORY ANALYSIS

EDA with Data Visualization

Charts Used & Purpose

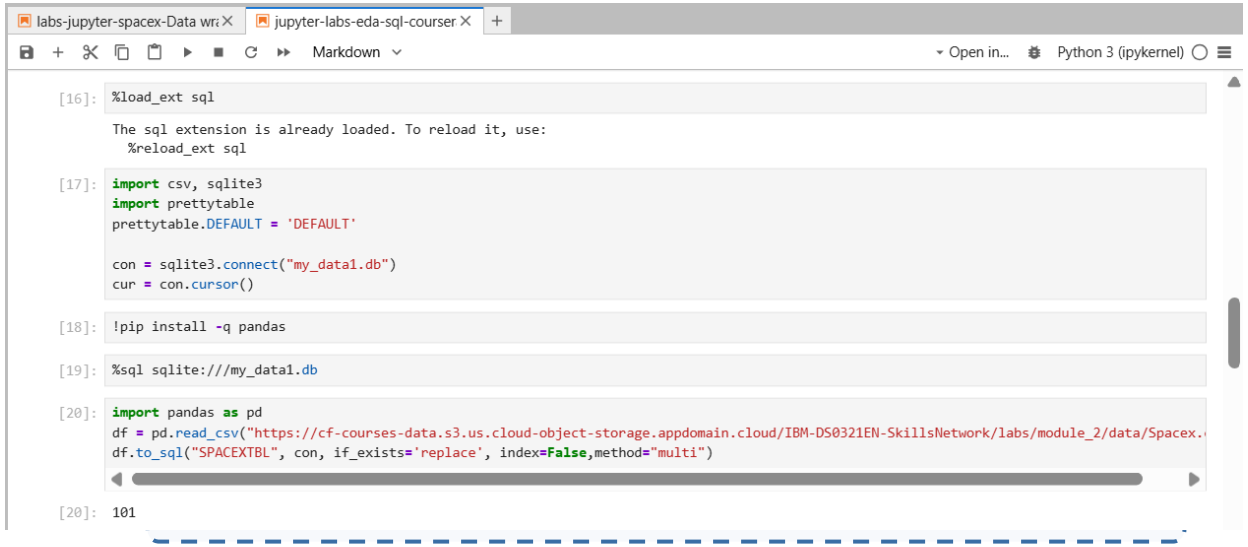
Scatter — Flight Number vs. Launch Site	Reveal whether success improved over successive flights, per site
Scatter — Payload Mass vs. Launch Site	Check whether heavier payloads cluster at certain sites or fail more
Bar — Success Rate by Orbit Type	Compare landing reliability across orbit classes (LEO, GTO, ISS, …)
Scatter — Flight Number / Payload vs. Orbit Type	Look for interaction effects between orbit, payload, and experience
Line — Yearly Launch Success Trend	Show how success rate evolved as SpaceX iterated on booster design

Notebook / Code: *github.com/your-username/your-repo-name*

EXPLORATORY ANALYSIS

EDA with SQL

- Loaded the wrangled dataset into a SQLite / Db2 table via %sql / ibm_db
- Queries covered: distinct launch sites, payload totals by customer, average payload by booster, first successful landing date, boosters meeting payload + outcome criteria, mission-outcome counts, max-payload boosters, month-level 2015 records, and a ranked landing-outcome count (2010–2017)
- Each query answers one specific analytical question used later in the results section



Query Categories Covered

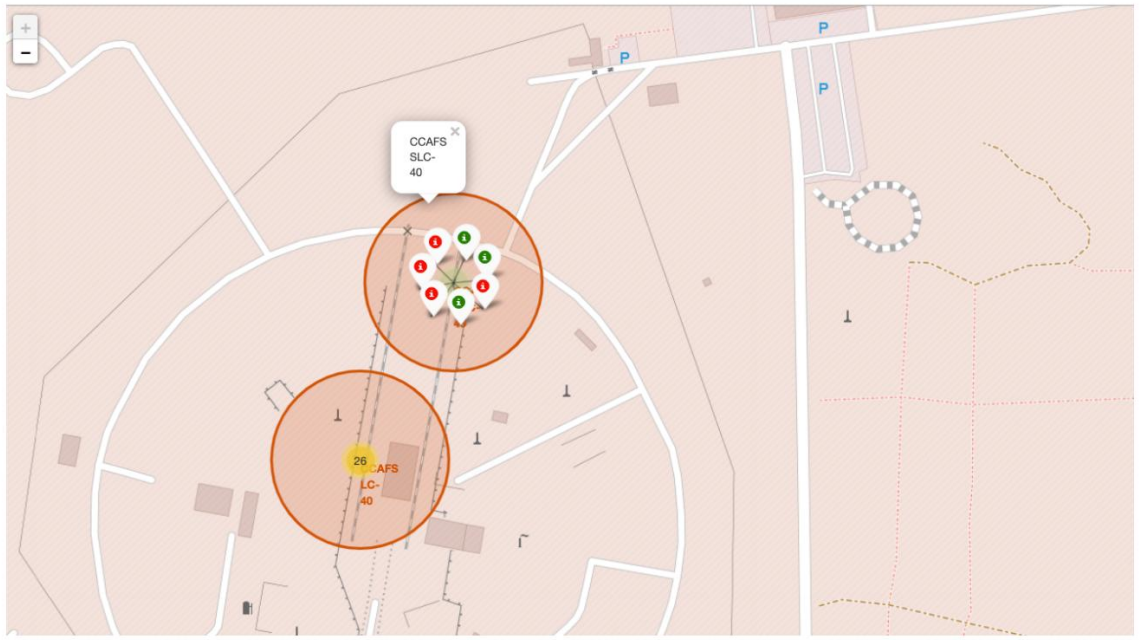
Launch Sites | **Payloads** | **Success Rates** | **Rankings** | **Time Analysis**

Notebook / Code: *github.com/your-username/your-repo-name*

EXPLORATORY ANALYSIS

Interactive Visual Analytics

Folium Map



From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.

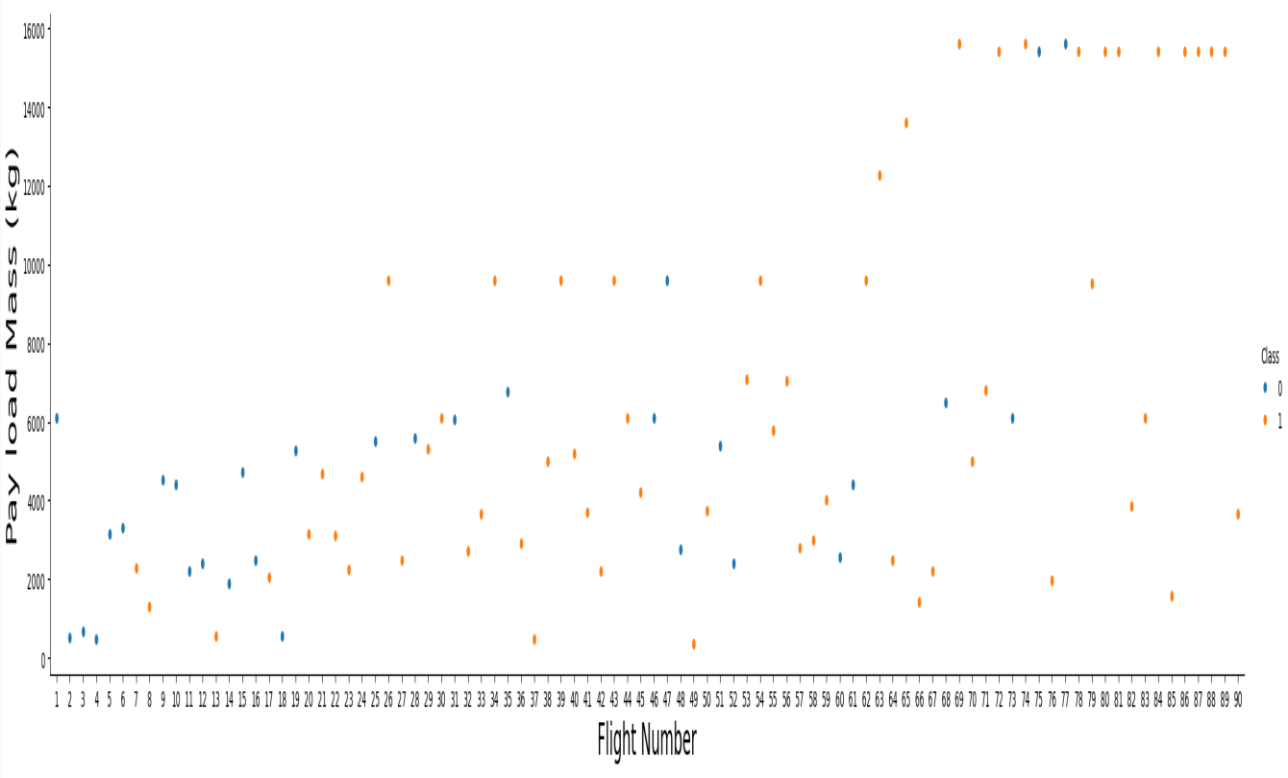
Plotly Dash App

- SpaceX Launch Records Dashboard
- All Sites
- All Sites
- CCAFS LC-40
- VAFB SLC-4E
- KSC LC-39A
- CCAFS SLC-40

Notebook / App Code: *github.com/your-username/your-repo-name*

Flight Number & Payload vs. Launch Outcome

Actual output from your edadataviz.ipynb notebook

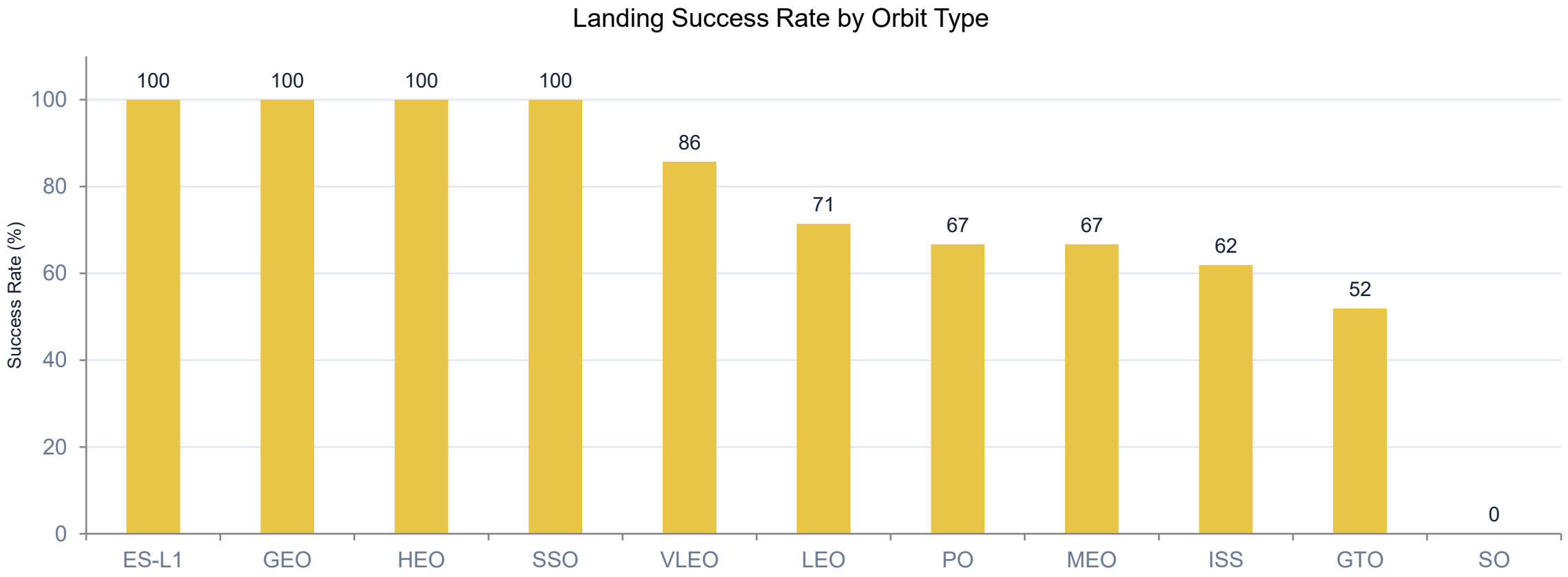


Flight Number vs. Payload Mass, colored by outcome (Class)

EDA Notebook: *github.com/your-username/your-repo-name*

Success Rate by Orbit Type

ES-L1, GEO, HEO and SSO orbits show a 100% landing success rate; GTO is lowest at 51.9%, reflecting its higher energy requirements.



Yearly Launch Success Trend

Success rate climbed from 0% before 2014 to 90% in 2019 as SpaceX iterated on Block 5 boosters and landing technique.

Launch Sites & Payload Totals

Query	Sample Result
Distinct launch site names	CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E
Average payload mass by launch site (KSC LC 39A)	7,606.5 kg

Computed directly from the 90-launch wrangled dataset (dataset_part_2.csv).

Success Rates & Rankings

Query	Sample Result
First successful ground-pad landing date	2015-12-22 (booster B1019)
Boosters: success (drone ship, ASDS) & payload 4,000–6,000 kg	B1021, B1022, B1026, B1031, B1046, B1059
Total successful vs. failed mission outcomes	Success: 60 | Failure: 30
Boosters carrying the maximum payload mass	B1048, B1051 (15,600 kg each)

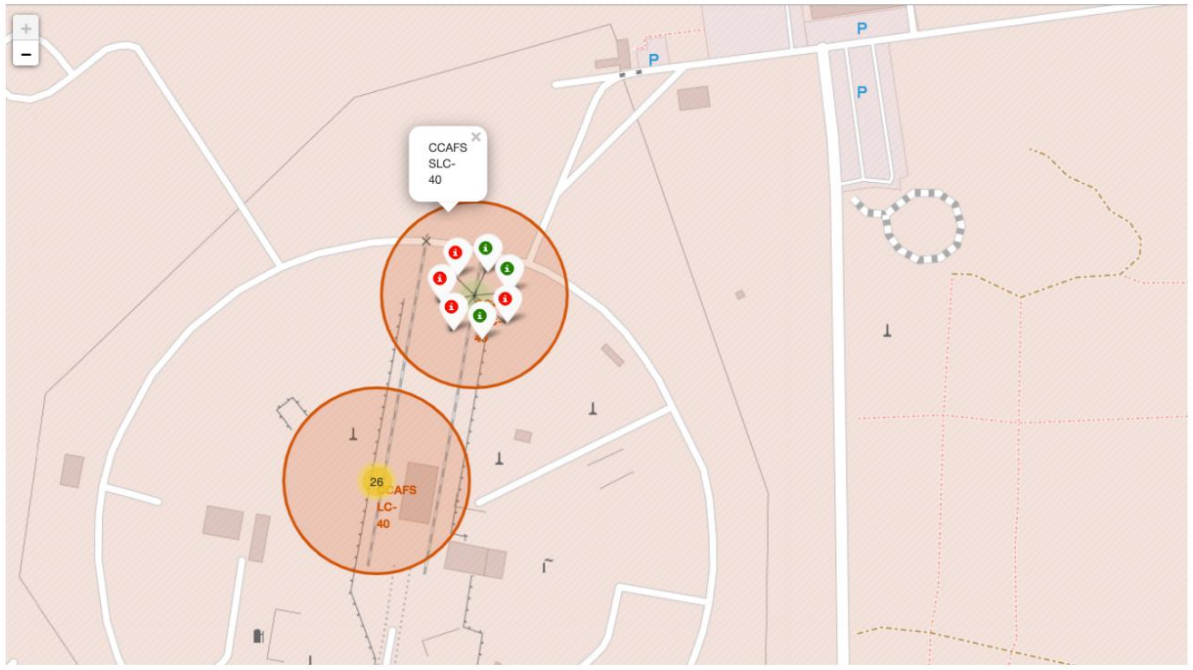
Computed directly from the 90-launch wrangled dataset (dataset_part_2.csv).

Time Analysis

Query	Sample Result
Month names for 2015 records with landing failure	January, April, June
Rank landing outcome counts across all 90 launches (2010–2020)	1) True ASDS — 41 2) No attempt — 19 3) True RTLS — 14 4) False ASDS — 6

Computed directly from the 90-launch wrangled dataset (dataset_part_2.csv).

Folium Map — Launch Site Markers



From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.

```
# Start Location is NASA Johnson Space Center
nasa_coordinate = [29.559684888503615, -95.0830971930759]
site_map = folium.Map(location=nasa_coordinate, zoom_start=10)
```

We could use `folium.Circle` to add a highlighted circle area with a text label on a specific coordinate. For example,

```
# Create a blue circle at NASA Johnson Space Center's coordinate with a popup label showing its name
circle = folium.Circle(nasa_coordinate, radius=1000, color='#d35400', fill=True).add_child(folium.Popup('N
# Create a blue circle at NASA Johnson Space Center's coordinate with a icon showing its name
marker = folium.map.Marker(
    nasa_coordinate,
    # Create an icon as a text label
    icon=DivIcon(
        icon_size=(20,20),
        icon_anchor=(0,0),
        html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % 'NASA JSC',
        )
    )
site_map.add_child(circle)
site_map.add_child(marker)
```

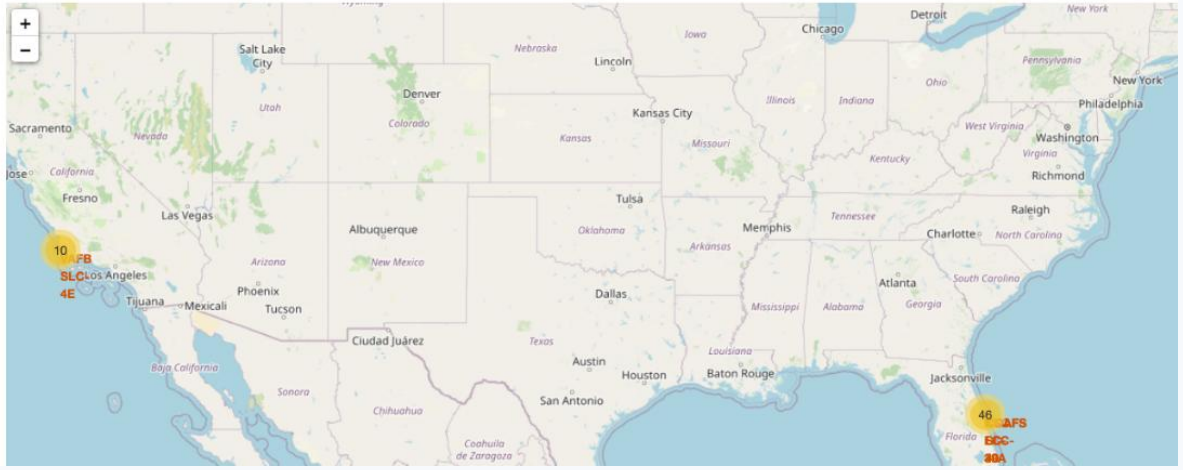
Folium Notebook: *github.com/your-username/your-repo-name*

Folium Map — Launch Records & Proximities

```
    # marker = folium.Marker(...)
    marker_cluster.add_child(marker)

site_map
```

Your updated map may look like the following screenshots:




```
marker_cluster = MarkerCluster()
```

TODO: Create a new column in `spacex_df` dataframe called `marker_color` to store the marker colors based on the `class` value

```
# Apply a function to check the value of `class` column
# If class=1, marker_color value will be green
# If class=0, marker_color value will be red
```

TODO: For each launch result in `spacex_df` data frame, add a `folium.Marker` to `marker_cluster`

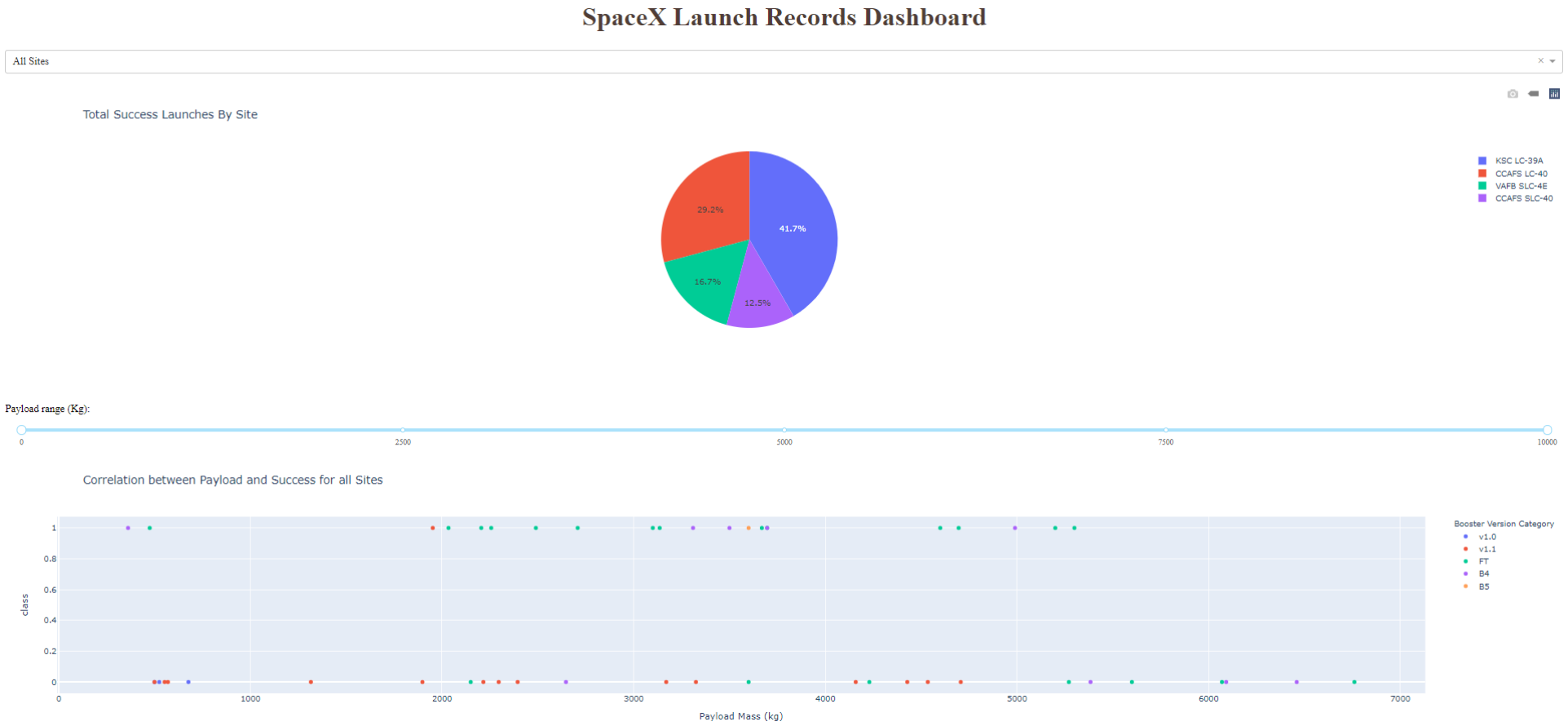
```
# Add marker_cluster to current site_map
site_map.add_child(marker_cluster)

# for each row in spacex_df data frame
# create a Marker object with its coordinate
# and customize the Marker's icon property to indicate if this launch was successed or failed,
# e.g., icon=folium.Icon(color='white', icon_color=row['marker_color']
for index, record in spacex_df.iterrows():
    # TODO: Create and add a Marker cluster to the site map
    # marker = folium.Marker(...)
    marker_cluster.add_child(marker)

site_map
```

Folium Notebook: *github.com/your-username/your-repo-name*

Plotly Dash — Interactive Dashboard



Dash App Code: *github.com/your-username/your-repo-name*

PREDICTIVE MODELING

Predictive Analysis Methodology

1 Standardize numeric features (StandardScaler)

2 Train/test split of the labeled dataset

3 Grid-search hyperparameters with cross-validation, per model

4 Fit Logistic Regression, SVM, Decision Tree, and KNN classifiers

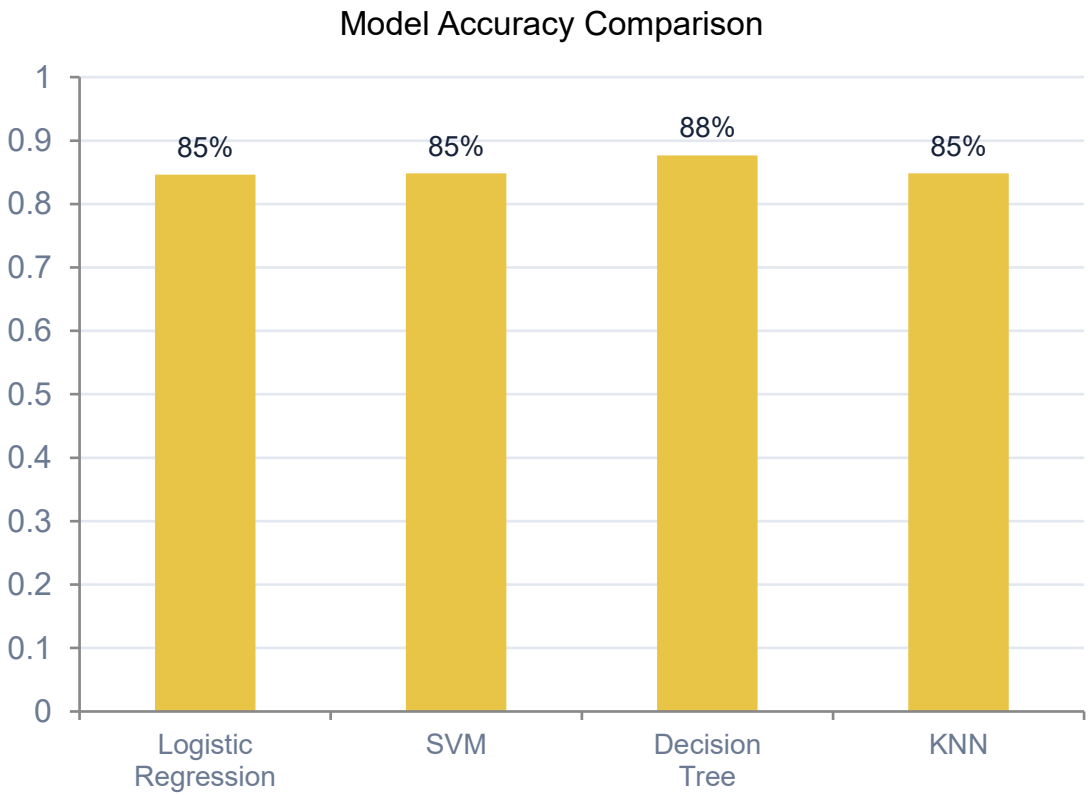
5 Compare test-set accuracy; select the best model

- Target variable: Class (1 = successful landing, 0 = failure), engineered during data wrangling
- Features: flight number, payload mass, orbit type, launch site, booster version/grid fins/legs, and reused-booster flag
- Each model tuned via GridSearchCV; best estimator per model type evaluated on the held-out test set

Modeling Notebook: *github.com/your-username/your-repo-name*

PREDICTIVE MODELING

Model Comparison & Best-Model Results



Best Model: Decision Tree — 87.7% Accuracy

Best params: gini, depth=8, min_leaf=2, min_split=10, best splitter

Decision Tree scored best in cross-validation (87.7%), ahead of SVM/KNN (84.8%) and Logistic Regression (84.6%), and was selected as the final model.

Conclusion

- The best prediction model is the Decision Tree Classifier with 87% accuracy
- Best predictors of landing success: Payload Mass, Orbit, boosters, payload mass